

Laboratorio di Programmazione

Lezione 3-4

Analisi dettagliata di un programma

`/* commento */`

Possono estendersi su più linee apparire in qualsiasi parte del programma.

Alternativamente

`// commento`

in questo caso possono estendersi su una sola linea.

```
/*Calcolo area rettangolo*/  
#include <stdio.h>
```

```
int main(void)  
{  
    int base;  
    int altezza;  
    int area;  
  
    base = 3;  
    altezza = 7;  
    area = base*altezza;  
  
    printf("%d\n", area);  
}
```

Variabili e assegnamenti

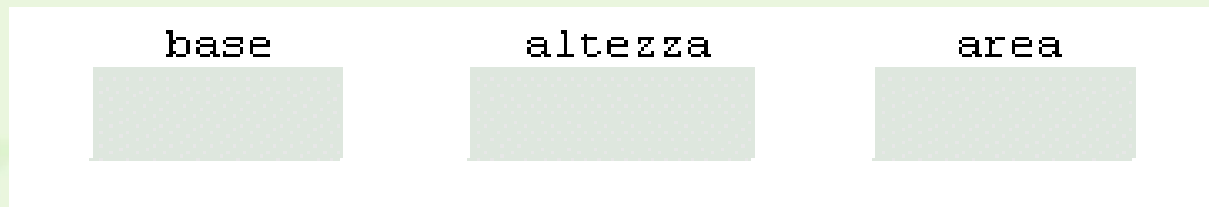
Dichiarazioni di variabili intere

```
int base;  
int altezza;  
int area;
```

`int` specifica che si tratta di una variabile numerica di tipo **intero**.

Come le altre istruzioni devono terminare con un punto e virgola.

Nell'esempio, a ogni dichiarazione di variabile corrisponde anche la sua **definizione**: le viene riservato uno spazio adeguato in memoria centrale:



Variabili e assegnamenti

Il **nome** di una variabile la identifica

il suo **tipo** ne definisce la dimensione e l'insieme delle operazioni che si possono effettuare su di essa.

La dimensione può variare rispetto all'implementazione

```
base = 3;
```

assegna alla variabile **base** il valore 3.

L'**assegnamento** è dunque realizzato mediante l'operatore **=**.

base	altezza	area
3	7	

Variabili e assegnamenti

```
area = base*altezza;
```

assegna alla variabile `area` il prodotto dei valori di `base` e `altezza`.

base	altezza	area
3	7	21

mentre l'ultima istruzione

```
printf("%d\n", area);
```

visualizza 21, il valore della variabile `area`.

% specifica che il carattere seguente (`d`) definisce il formato di stampa di `area`.

`d` (***decimal***) indica il formato di stampa: un intero nel sistema decimale.

`\n` provoca un salto a linea nuova dopo la visualizzazione.

Input e output

Per immettere valori:

```
scanf ("%d", &base);
```

%d indica un valore intero in formato decimale assegnato alla variabile **base**. **&base** indica l'indirizzo di memoria in cui si trova la variabile **base**.

```
printf("Valore della base: ");
```

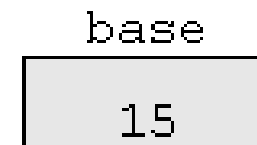
```
scanf ("%d", &base);
```

Quello che apparirà all'*utente* sarà:

Valore della base:

se l'utente digita 15:

Valore della base: 15 Invio



questo verrà assegnato alla variabile **base**

Input e Output

Nell'ipotesi che l'utente inserisca i valori 10 e 13 avremo:

AREA RETTANGOLO

Valore base: 10

Valore altezza:13

Base: 10

Altezza: 13

Area: 130

```
/* Calcolo area rettangolo */
#include <stdio.h>

int main(void){
    int base, altezza, area;

    printf("AREA RETTANGOLO\n\n");
    printf("Valore base: ");
    scanf("%d", &base);
    printf("Valore altezza: ");
    scanf("%d", &altezza);

    area = base*altezza;

    printf("Base: %d\n", base);
    printf("Altezza: %d\n", altezza);
    printf("Area: %d\n", area);
}
```

Input e output

Per visualizzare più variabili con una sola `printf`

```
printf("%d %d %d", base, altezza, area);
```

degli a-capo a piacere:

```
printf("%d\n%d\n%d", base, altezza, area);
```

del testo a chiarificare:

```
printf("Base: %d\nAltezza: %d\nArea: %d", base, altezza, area);
```

Input e output

Sequenze di escape, sequenze di caratteri con funzioni speciali:

- `\n` va a linea nuova
- `\t` salta di una tabulazione
- `\b` ritorna un carattere indietro (*backspace*)
- `\a` suona il campanello della macchina
- `\\` stampa il carattere `\`
- `\"` stampa il carattere `"`

Ricominciamo ...

Struttura generale di un programma

```
inclusioneDiLibrerie
```

```
int main(void)
{
    dichiarazioneDiVariabili
    istruzione1
    istruzione2
    istruzione3
    ...
    istruzioneN
}
```

NOTA: Nella sintassi il punto e virgola fa parte dell'istruzione stessa.

Identificatori

- Nomi per le variabili (e le funzioni)
- Scelti dal programmatore
- Possono contenere numeri e lettere maiuscole e minuscole (sono diverse!) e underscore _
- Devono iniziare per lettera
(di solito minuscola)
- Non devono essere parole riservate o nomi di funzione

Dichiarazioni di variabili

- Ogni variabile DEVE essere dichiarata prima del suo utilizzo.
- Sintassi:
 <tipo> <identificatore>;
 <tipo> <identif_1>, ... <identif_k>;
- Possono contenere numeri e lettere maiuscole e minuscole (sono diverse!) e underscore _
- Devono iniziare per lettera

Esempio

```
int num, somma;  
int valore1;  
char risposta;  
float val_1, val_2;
```

Operatore cast

```
int num1, num2;  
  
float media;  
  
media = (float) (num1 + num2) / 2;
```

Applica una conversione di tipo al risultato della divisione.

Operatore di Assegnamento

- Si utilizza il **simbolo =**
- Il valore alla destra di = viene assegnato alla variabile a sinistra di =
- Esempi:

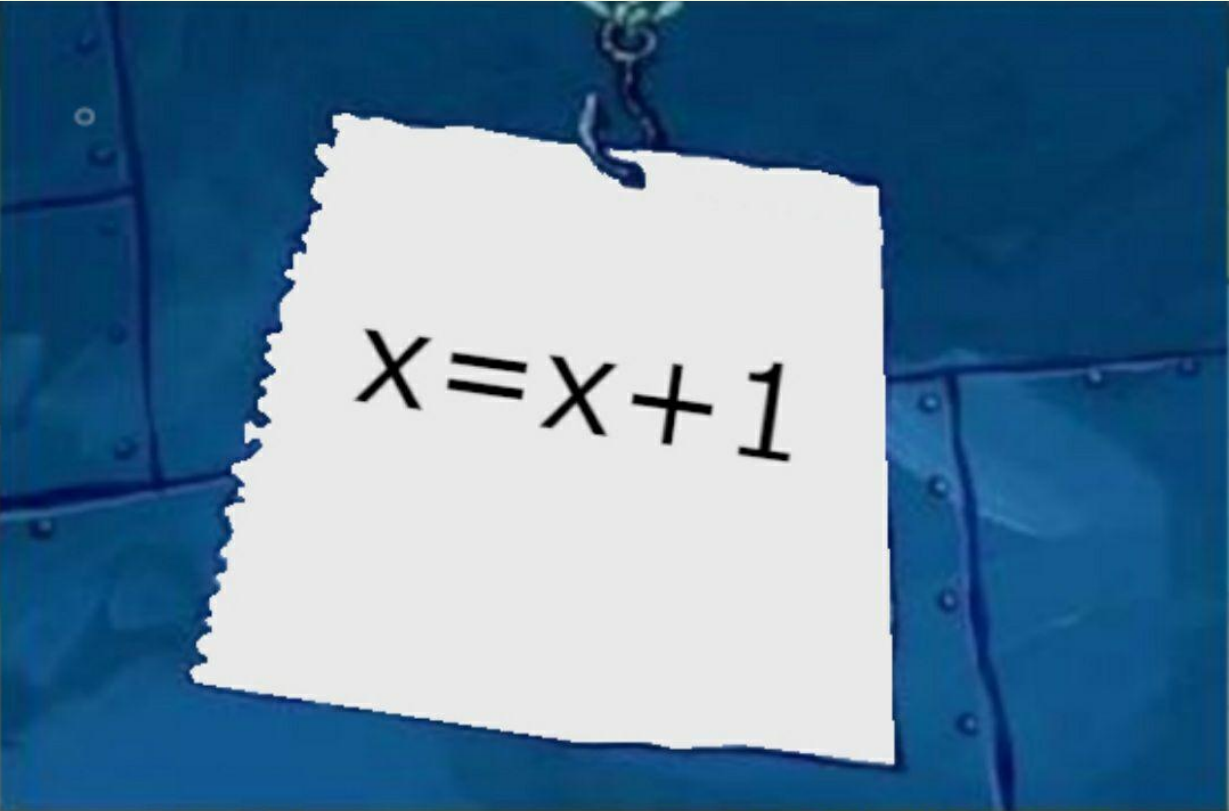
```
num = 57;  
valore = 34,5;  
risp = 's';  
  
num = valore;
```

Operatori Aritmetici

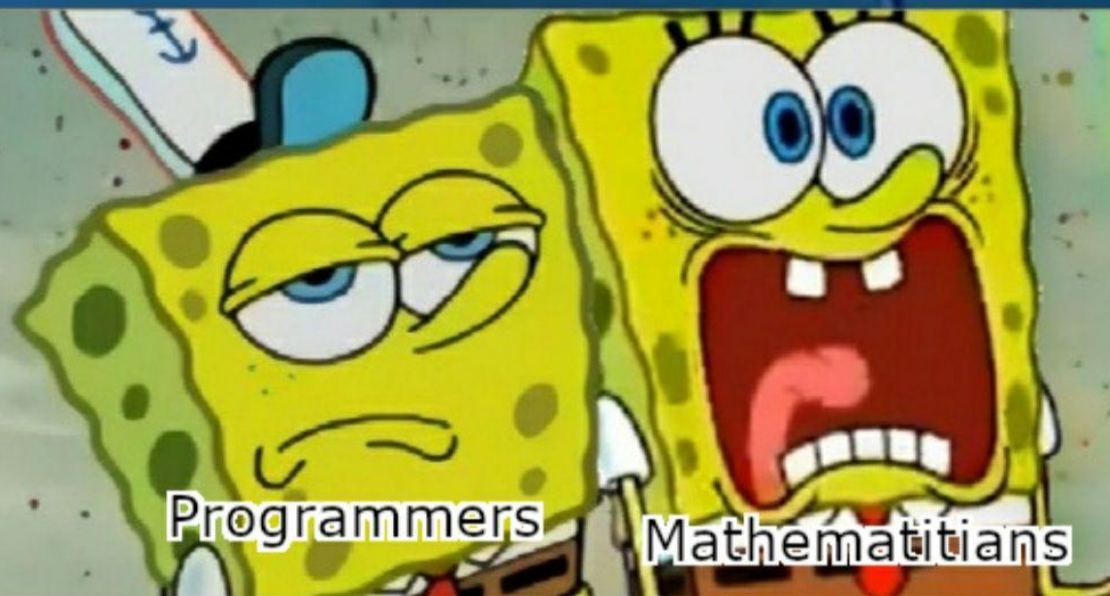
- **Espressioni aritmetiche** sono formate usando gli operatori **+**, **-**, *****, **/**, **%** insieme a variabili e numeri (*operandi*). Il risultato è un numero.
 - Se entrambi gli operandi sono dello stesso tipo, il risultato è di quel tipo.
 - Le operazioni *****, **/**, **%** hanno priorità rispetto a **+**, **-**.
 - L'ordine di priorità può essere alterato usando le parentesi **(,)**.

- Esempi:

```
num = (57 + 3) * 23 ;  
num = num +1 ;
```



$x = x + 1$



Programmers

Mathematitians

Operatori di uguaglianza e relazionali

- Sono operatori che applicati a variabili di "tipo numerico" (o **char**) danno come risultato un valore booleano **true** o **false**: <, >, <=, >=, ==, !=
- Esempi:

```
num > 5;  
n1 + n2 >= 14;  
valore == n1;  
continua = val==3;
```

Operatori booleani

- Sono operatori che applicati a variabili di tipo booleano (**true** o **false**) restituiscono un valore booleano:

&& (and) **||** (or) **!** (not)

- Esempi:

```
(num > 5) && (i!=0) ;  
! (n1 + n2 >= 14) ;  
n1 || n2 ;  
val = risp && (r=='s') ;
```

ATTENZIONE:

- In C non esiste il tipo **boolean**, ogni numero non 0 è interpretato come **true** mentre lo 0 corrisponde a **false** .
- Ad esempio:

num && (a<5)

è equivalente a

(num!=0) && (a<5)

Espressioni booleane (condizioni logiche)

- Sono espressioni composte da variabili, espressioni aritmetiche e operatori relazionali, di uguaglianza e booleani che hanno come risultato un valore logico (**true** o **false**) .
- Si utilizzano nelle **strutture di controllo condizionali** e **iterative**.

Regole di precedenza

Più alta

- Operatori *unari* $+$, $-$, $!$
- Operatori aritmetici *binari* $*$, $/$, $\%$
- Operatori aritmetici binari $+$, $-$
- Operatori relazionali $<$, $>$, $\leq$, $\geq$
- Operatori di uguaglianza $==$, $!=$
- Operatore booleano $\&\&$
- Operatore booleano $||$
- Operatore di assegnamento $=$

Più bassa

Esempi

```
int x = 5;
```

```
int y = 10, z = 15;
```

```
b = (x < 5 && y > x);
```

```
b = (x < 5 || y > x);
```

```
b = (x > 3 || y < 10 && z == 15);
```

```
b = (! (x > 3) && x != z || x + y == z);
```

L'istruzione **if**

- Sintassi

if (*<valore booleano>*)

<istruzione_1>

<istruzione_2>

<istruzione_3>

.....

- Significato

<istruzione_1> viene eseguita soltanto se *<valore booleano>* è *true*

Esempio

- Visto oggi in laboratorio :

```
if (a < b)
    min=a;
if (b < a)
    min=b;
printf("il minimo e' %d",min);
.....
.....
```

Istruzioni composte (blocchi)

- Quando un gruppo di istruzioni è racchiuso tra coppia di parentesi graffe (**{}**), esse formano una unica *istruzione composta*.
- Sintassi:

{

Istruzione_1

Istruzione_2

...

}

Esempio

- Uso istruzione composta per eseguire diverse istruzioni dentro un **if**

```
if (cont < 3)
{
    totale = 0;
    printf("tutto sbagliato!");
}
prezzo = 50;
.....
.....
```

L'istruzione **if**

- Sintassi

```
if ( <espressione booleana> ){  
    <istruzione_1>  
    .....  
    <istruzione_r>  
}  
  
<istruzione_x>  
<istruzione_y>  
.....
```

- Significato

Il blocco <istruzione_1>..*<istruzione_r>* viene eseguito **soltanto** se <espressione booleana> è *true*

Attenzione.... Errori in vista!

```
if ( num1 = num2 )  
    printf( "%d e' uguale a %d\n", num1, num2 ); NO!
```

```
if ( num1 == num2 );  
    printf( "%d e' uguale a %d\n", num1, num2 ); NO!
```

```
if ( num1 == num2)  
    printf( "%d e' uguale a %d" \n, num1, num2 ); NO!
```

```
if ( num1 == num2)  
    printf( "&d e' uguale a &d \n ", num1, num2 ); NO!
```

```
if ( num1 == num2 )  
    printf( "%d e' uguale a %d\n", num1, num2 ); OK!
```

L'istruzione **if...else**

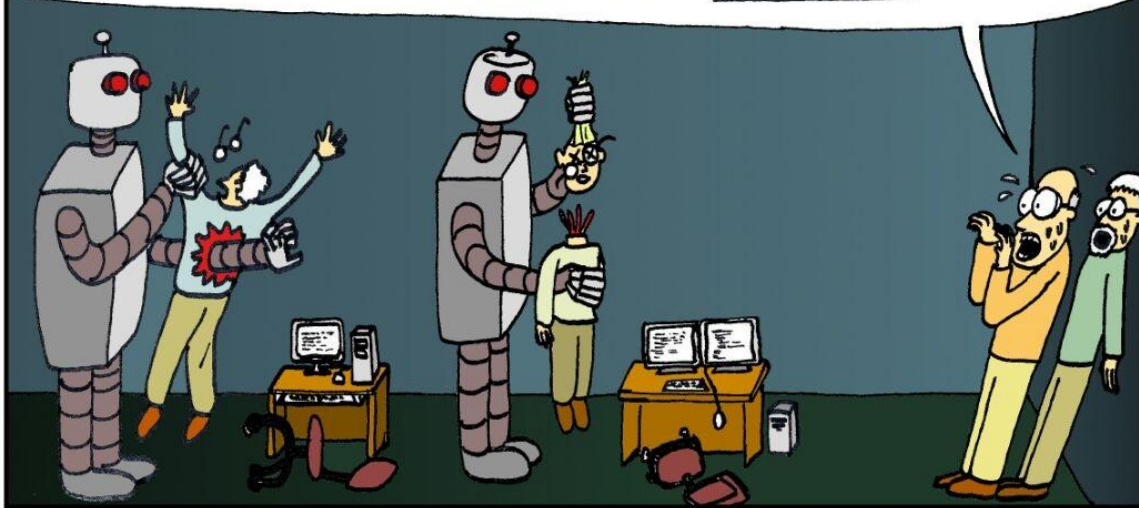
- Sintassi

```
if ( <espressione booleana> )
{ <istruzione_1>;
  .....
  <istruzione_r>; }
else
{ <istruzione_r+1>;
  .....
  <istruzione_s>; }
<istruzione_x>;
<istruzione_y>;
.....
```

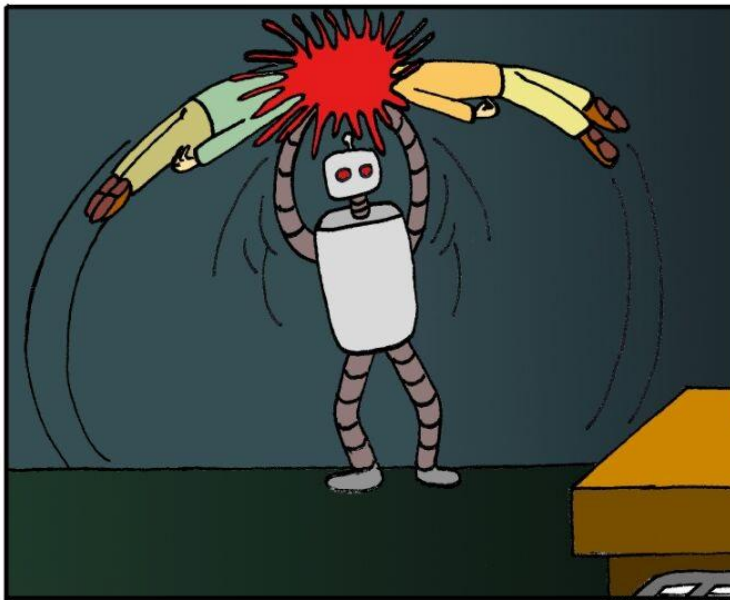
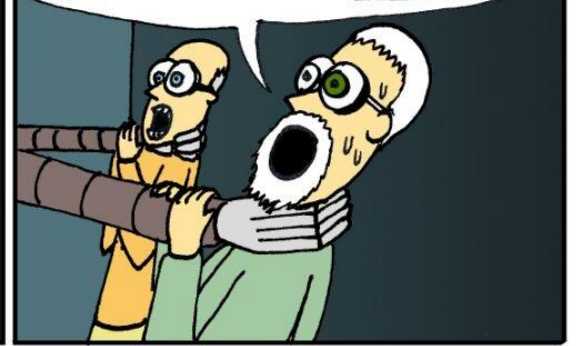
- Significato

SE <espressione booleana> è **true** viene eseguito Il blocco
<istruzione_1>..*<istruzione_r>* **ALTRIMENTI** viene eseguito Il blocco
<istruzione_r+1>..*<istruzione_s>;*

OH NO! THE ROBOTS ARE KILLING US!!!



BUT WHY?!? WE NEVER PROGRAMMED THEM TO DO THIS!!!



```
main() {  
    int isCrazyMurderingRobot=0;  
  
    if (isCrazyMurderingRobot=1)  
        kill(humans);  
    else  
        be_nice_to(humans);  
}
```

Esercizio

```
if (a%3==0 || a%2==0)
    printf("%d", 2*a);
else
    printf("%d", 5*a);
```

- Se a=10 il programma stampa ?
- Se a=42 il programma stampa ?
- Se a=18 il programma stampa ?
- Se a=25 il programma stampa ?

Istruzione **while**

- Sintassi

```
while (<espressione booleana>){  
    <istruzione_1>  
    ...  
    <istruzione_r>  
}  
  
<istruzione_x>;  
<istruzione_y>;  
.....
```

- Significato

Il blocco *<istruzione_1>..<istruzione_r>* viene eseguito **fintanto che** *<espressione booleana>* è *true*

Iterazione definita : ciclo con "contatore"

numero di iterazioni conosciuto a-priori

- Utilizzo una variabile per contare:
 - La inizializzo a 0 prima del ciclo
 - All'interno del ciclo la incremento
 - La condizione del while controlla quando arrivo al valore scelto

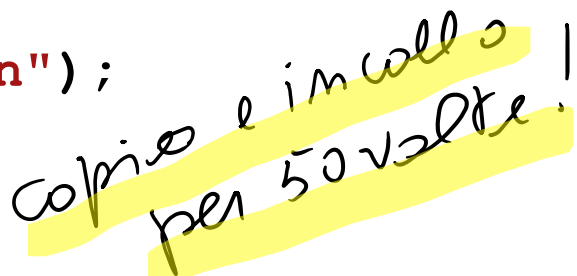
```
cont =0;
while (cont < 5)
{
    printf("finalmente una ripetizione! \n");

    cont= cont+1;
}
printf("tutto bene?");

.....
```

Esercizio: Calcolare la somma di 50 numeri immessi dall'utente.

```
int a, s;  
s=0;  
printf("scrivi il numero\n");  
scanf("%d", &a);  
s=s+a;  
  
printf("scrivi il numero\n");  
scanf("%d", &a);  
s=s+a;  
  
printf("scrivi il numero\n");  
scanf("%d", &a);  
s=s+a;  
.....  
.....  
printf("la somma dei numeri è %d", s);
```



Esercizio : Calcolare la somma di 50 numeri immessi dall'utente.

muovo!

```
int a, s, c;
```

```
s=0;
```

```
c=0;
```

```
while( c < 50 ){
```

```
    printf("scrivi il numero\n");
```

```
    scanf("%d", &a);
```

```
    s=s+a;
```

```
    c=c+1;
```

```
}
```

```
printf("la somma dei numeri è %d", s);
```

c è il mio contatore